Software Testing and Validation

Second Project — Report

Dinis Rodrigues (ist106213) Artur Krystopchuk (ist104145)
Gabriel Santos (ist1116334)

5 June 2026

Abstract

This report presents the design of test cases for the urban delivery system described in the specification of the second project for the Software Testing and Validation course. Appropriate design strategies are applied to test the Order and Delivery classes at the class level, as well as the `computeDeliveryCost()` and `updateItem()` methods. The final section presents the TestNG implementation of eight selected test cases from the test suite of the Order class.

Contents

1 Class-Level Testing of the Order Class **2**
1.1 Chosen Strategy: Non-Modal Class Testing . . . 2
1.2 On/Off Points . . . 2
1.3 Domain Matrix with Test Vectors . . . 2
1.4 Relational Invariant (Same Supplier) . . . 4

2 Class-Level Testing of the Delivery Class **4**
2.1 State Machine . . . 4
2.2 Transition tree . . . 4
2.3 Testing strategy: Modal Class Testing . . . 5
2.4 Test Cases . . . 6

3 Method `computeDeliveryCost()` **7**
3.1 Strategy: Decision Table / Boundary Analysis . . . 7
3.2 Decision Table . . . 7
3.3 Test Cases . . . 7

4 Method `updateItem()` **7**
4.1 Chosen Strategy: Combinational State-Input Testing & Boundary Analysis . . . 7
4.2 State + Input Combinations Matrix . . . 8
4.3 Test Cases Description . . . 8

5 Implementation in TestNG **9**

1 Class-Level Testing of the **Order** Class

1.1 Chosen Strategy: Non-Modal Class Testing

The Order class does not possess a state machine (there are no modes that alter method behaviour); its class-level behaviour is determined by a set of *invariants* over its instance variables. The appropriate strategy for testing a class of this type is **Non-Modal Class Testing**, which consists of: (i) identifying the domains of the variables; (ii) determining the *On*/*Off* points for each invariant boundary; and (iii) constructing a *domain matrix* with test vectors, where each test case exercises one boundary at a time, keeping the remaining variables at typical values.

1.2 On/Off Points

Boundary (invariant)	**On**	**Off**
client $\neq$ `null`	Valid client	`null`
addr $\neq$ `null`	Valid string	`null`
addrLength ≥ 1	1	0 (empty)
distance ≥ 0	0	-1
distance ≤ 20000	20000	20001
quantity ≥ 1	1	0
quantity ≤ 10	10	11
cost ≤ 1000	1000	1000.01
totalQty $\leq 20 + \lfloor \text{cost}/50 \rfloor$	= limit	limit + 1

1.3 Domain Matrix with Test Vectors

Each column represents a test case. In each column, exactly one boundary is in *On* or *Off* (shaded cell), while the remaining variables are at a typical value (row *In*). Adopted typical value: order with one item of price 50 and quantity 4 (cost = 200, totalQty = 4, limit = 24), address length of 3, and distance = 5000.

Table 2: Domain matrix for the Order class.

Var.	Condition		1	2	3	4	5	6	7	8	9	10
addr	≥ 1	On	1									
		Off		0								
	Typical	In			3	3	3	3	3	3	3	3
distance	≥ 0	On			0							
		Off				−1						
	≤ 20000	On					20000					
		Off						20001				
	Typical	In	5000	5000					5000	5000	5000	5000
quantity	≥ 1	On							1			
		Off								0		
	≤ 10	On									10	
		Off										11
	Typical	In	4	4	4	4	4	4				
cost	≤ 1000	On										
		Off										
	Typical	In	200	200	200	200	200	200	50	0	500	550
totalQty	$\leq$ lim.	On										
		Off										
	Typical	In	4	4	4	4	4	4	1	0	10	11
Expected result			✓	✗	✓	✗	✓	✗	✓	✗	✓	✗

Table 3: Domain matrix for the Order class (continuation).

Var.	Condition		11	12	13	14	15	16	17	18
client	$\neq$ `null`	On					ok			
		Off						`null`		
	Typical	In	ok	ok	ok	ok			ok	ok
addr	$\neq$ `null`	On							`"x"`	
		Off								`null`
	typical length	In	3	3	3	3	3	3		
distance	Typical	In	5000	5000	5000	5000	5000	5000	5000	5000
quantity	Typical	In	—	—	—	—	—	—	—	—
cost	≤ 1000	On	1000							
		Off		1000.01						
	Typical	In			200	200	200	200	200	200
totalQty	$\leq$ lim.	On			24					
		Off				25				
	Typical	In	20	20			4	4	4	4
Expected result			✓	✗	✓	✗	✓	✗	✓	✗

Legend: ✓ = valid state (operation accepted); ✗ = invalid state (must throw `InvalidOperatio`

`nException` without changing the order state, according to I6).

1.4 Relational Invariant (Same Supplier)

This invariant is non-numeric, and therefore does not have On/Off points. It is tested with two dedicated cases:

#	Scenario	Expected Result
19	Add item from supplier S to an order already containing items from supplier S	Accepted (✓)
20	Add item from supplier S' to an order already containing items from supplier S	`InvalidOperationException` (✗)

2 Class-Level Testing of the **Delivery** Class

2.1 State Machine

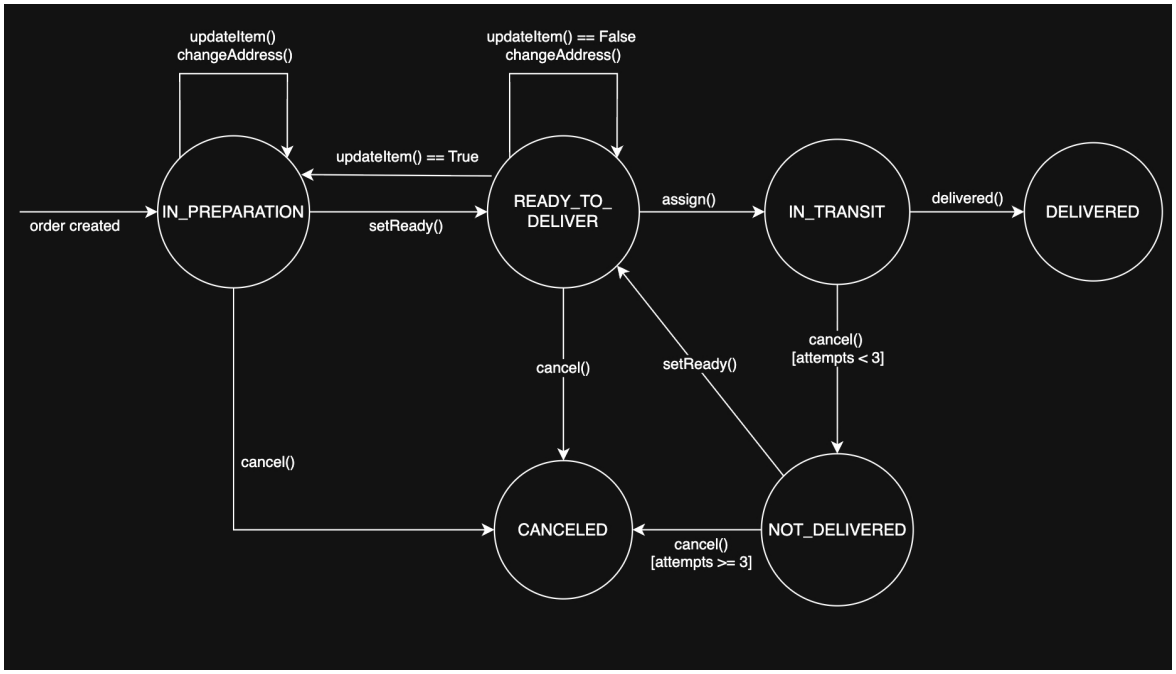


Figure 1: State machine of the Delivery class.

2.2 Transition tree

The transition tree is obtained by unfolding the state machine breadth-first from the initial state, stopping the expansion whenever a state has already appeared on the path from the root (*round-trip* criterion). Each path from the root to a leaf corresponds to one test case.

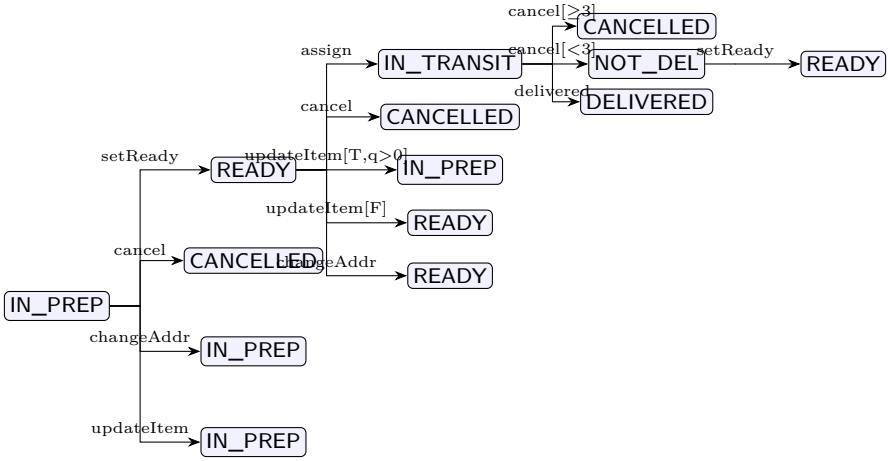


Figure 2: Transition tree of the Delivery class.

The paths from the root to the leaves yield 14 test cases for valid transitions (Table 5).

Table 5: Test paths derived from the transition tree.

TC	Event sequence	Final state
1	new	IN_PREPARATION
2	new → updateItem	IN_PREPARATION
3	new → changeAddress	IN_PREPARATION
4	new → cancel	CANCELLED
5	new → setReady	READY_TO_DELIVER
6	new → setReady → cancel	CANCELLED
7	new → setReady → changeAddress	READY_TO_DELIVER
8	new → setReady → updateItem [false]	READY_TO_DELIVER
9	new → setReady → updateItem [true, qty>0]	IN_PREPARATION
10	new → setReady → assign	IN_TRANSIT
11	new → setReady → assign → delivered	DELIVERED
12	new → setReady → assign → cancel [att<3]	NOT_DELIVERED
13	new → setReady → assign → cancel [att≥3]	CANCELLED
14	new → setReady → assign → cancel [<3] → setReady	READY_TO_DELIVER

2.3 Testing strategy: Modal Class Testing

Unlike the Order class, the Delivery class has an explicit *state machine*: its behaviour depends on the mode the delivery is in (`IN_PREPARATION`, `READY_TO_DELIVER`, `IN_TRANSIT`, `DELIVERED`, `NOT_DELIVERED`, `CANCELLED`), and each method is valid only in a subset of these modes. The class exhibits two kinds of constraint:

- **Sequence constraints** — the validity of a method invocation depends on the sequence of prior events (for example, `assign()` is only valid after `setReady()` has been called, which is captured by the current mode);
- **Domain constraints** — the validity (or outcome) of a method depends on the current content of the object (for example, `assign()` additionally requires the courier to have fewer than 5 deliveries assigned; `cancel()` from `IN_TRANSIT` branches on the number of delivery

attempts; `updateItem()` from `READY_TO_DELIVER` returns to `IN_PREPARATION` only if the resulting quantity is positive).

The coexistence of these two kinds of constraint is the defining feature of a *modal* class, and the appropriate strategy is **Modal Class Testing**.

2.4 Test Cases

Table 6: Final test suite for the Delivery class.

TC	Event sequence	Final state	Exception?
Valid transitions (from the transition tree)			
1	new	in_preparation	No
2	new → updateItem	in_preparation	No
3	new → changeAddress	in_preparation	No
4	new → cancel	cancelled	No
5	new → setReady	ready_to_deliver	No
6	new → setReady → cancel	cancelled	No
7	new → setReady → changeAddress	ready_to_deliver	No
8	new → setReady → updateItem [false]	ready_to_deliver	No
9	new → setReady → updateItem [true, qty>0]	in_preparation	No
10	new → setReady → assign	in_transit	No
11	new → setReady → assign → delivered	delivered	No
12	new → setReady → assign → cancel [att<3]	not_delivered	No
13	new → setReady → assign → cancel [att≥3]	cancelled	No
14	new → setReady → assign → cancel [<3] → setReady	ready_to_deliver	No
Invalid invocations from `IN_PREPARATION`			
15	new → assign	in_preparation	Yes
16	new → delivered	in_preparation	Yes
Invalid invocations from `READY_TO_DELIVER`			
17	new → setReady → setReady	ready_to_deliver	Yes
18	new → setReady → delivered	ready_to_deliver	Yes
Invalid invocations from `IN_TRANSIT`			
19	new → setReady → assign → setReady	in_transit	Yes
20	new → setReady → assign → assign	in_transit	Yes
21	new → setReady → assign → changeAddress	in_transit	Yes
22	new → setReady → assign → updateItem	in_transit	Yes
Invalid invocations from `NOT_DELIVERED`			
23	new → setReady → assign → cancel [<3] → assign	not_delivered	Yes

TC	Event sequence	Final state	Exception?
24	new → setReady → assign → cancel [<3] → delivered	not_delivered	Yes
25	new → setReady → assign → cancel [<3] → cancel	not_delivered	Yes
26	new → setReady → assign → cancel [<3] → changeAddress	not_delivered	Yes
27	new → setReady → assign → cancel [<3] → updateItem	not_delivered	Yes
Invalid invocations from `DELIVERED` (terminal)			
28	new → setReady → assign → delivered → setReady	delivered	Yes
29	new → setReady → assign → delivered → assign	delivered	Yes
30	new → setReady → assign → delivered → delivered	delivered	Yes
31	new → setReady → assign → delivered → cancel	delivered	Yes
32	new → setReady → assign → delivered → changeAddress	delivered	Yes
33	new → setReady → assign → delivered → updateItem	delivered	Yes
Invalid invocations from `CANCELLED` (terminal)			
34	new → cancel → setReady	cancelled	Yes
35	new → cancel → assign	cancelled	Yes
36	new → cancel → delivered	cancelled	Yes
37	new → cancel → cancel	cancelled	Yes
38	new → cancel → changeAddress	cancelled	Yes
39	new → cancel → updateItem	cancelled	Yes

3 Method `computeDeliveryCost()`

3.1 Strategy: Decision Table / Boundary Analysis

3.2 Decision Table

3.3 Test Cases

4 Method `updateItem()`

4.1 Chosen Strategy: Combinational State-Input Testing & Boundary Analysis

The behavior of the `updateItem()` method depends simultaneously on the current execution mode of the Delivery object, the properties of the input parameters, and an internal history tracking counter. Therefore, the chosen test design strategy is a combinational synthesis of **State-Based Contextual Constraints** and **Equivalence Partitioning / Boundary Value**

Analysis (BVA) applied to the input variables.

The strategy systematically checks:

1. **Contextual Constraints:** Allowed states (`IN_PREPARATION`, `READY_TO_DELIVER`) vs. forbidden states that must safely bypass execution and throw an exception.
2. **Input Constraints:** Item supplier compatibility, unit bounds ($1 \leq Q_{item} \leq 10$) , overall positive total quantity residual ($Q_{total} > 0$) , and negative input tracking for item subtraction.
3. **History Constraints:** Safe blocking execution after exactly 3 successful updates have transpired.

4.2 State + Input Combinations Matrix

Let U represent the cumulative count of successful `updateItem()` operations executed on a single instance ($0 \leq U \leq 4$). Let Q_{item} represent the requested item quantity modification, and let Q_{final_total} represent the prospective aggregated total units of all items combined in the delivery after update execution.

The logical vectors governing the input domain matrix partitions are mapped below:

Variable / Context	**Partition**	**Values / Conditions**	**Expected Outcome Category**
System Mode	Valid context	`IN_PREPARATION`, `READY_TO_DELIVER`	Process business rules
	Invalid context	Other FSM states	Throw `InvalidInvocationException`
Update Counter (U)	$U < 3$	$0, 1, 2$ updates achieved	Allowed to succeed if bounds valid
	$U \geq 3$	3 updates already processed	Operation fails (returns `false`)
Input Quantity (Q_{item})	Under Bound	≤ 0 (if subtraction results in total ≤ 0)	Operation fails (returns `false`)
	Valid Positive	$1 \leq Q_{item} \leq 10$	Success; changes state if ready
	Valid Negative	$-10 \leq Q_{item} \leq -1$ (with total > 0)	Success; mode unchanged
	Over Bound	> 10	Operation fails (returns `false`)
Total Residual (Q_{final_total})	Valid	> 0 units remaining	Success
	Invalid	≤ 0 units remaining	Operation fails (returns `false`)

4.3 Test Cases Description

The following specific test vectors systematically exercise the boundary combinations derived from the strategy:

TC #	Initial Mode	Prior U	Input Q_{item}	Expected Q_{final_total}	Expected Outcome / Side Effect
4.3.1	`IN_PREPARATION`	0	5	Valid (> 0)	Returns `true`. Mode remains `IN_PREPARATION`.
4.3.2	`READY_TO_DELIVER`	0	3	Valid (> 0)	Returns `true`. Mode reverts back to `IN_PREPARATION`.
4.3.3	`READY_TO_DELIVER`	1	-2	Valid (> 0)	Returns `true`. Mode remains `READY_TO_DELIVER`.
4.3.4	`IN_PREPARATION`	0	11	Valid (> 0)	Returns `false`. Bound violated ($Q_{item} > 10$). State unchanged.
4.3.5	`IN_PREPARATION`	2	-5	Invalid (≤ 0)	Returns `false`. Total residual unit breakdown (≤ 0). State unchanged.
4.3.6	`READY_TO_DELIVER`	3	2	Valid (> 0)	Returns `false`. Saturation limit met ($U = 3$). State unchanged.
4.3.7	`IN_TRANSIT`	0	4	Valid (> 0)	Throws `InvalidInvocationException`. State unchanged.
4.3.8	`CANCELLED`	0	1	Valid (> 0)	Throws `InvalidInvocationException`. State unchanged.

5 Implementation in TestNG

Four successful cases and four unsuccessful cases were selected from the test suite of the Order class (Section 1).

```
1  package supereats.tests;
2
3  import org.testng.annotations.*;
4  import static org.testng.Assert.*;
5
6  import supereats.core.*;
7
8  public class OrderTest {
9
10   private static final double EPS = 1e-9;
11   private Supplier s1;      // reference supplier
12   private Supplier s2;      // distinct supplier
13   private Client   client;
14   // Items from supplier s1 (weight in grams, integer price)
15   private Item i1;          // price 10
16   private Item i2;          // price  5
17   private Item i50;         // price 50 (for totalQty cases)
```

```
18
19    // Item from a distinct supplier (s2)
20    private Item iOther;
21
22    @BeforeMethod
23    public void setUp() {
24      s1     = new Supplier("Loja A");
25      s2     = new Supplier("Loja B");
26      client = newClient();
27
28      i1     = new Item(100,  10, s1, "Item 1");
29      i2     = new Item(200,   5, s1, "Item 2");
30      i50    = new Item(150,  50, s1, "Item 50");
31      iOther = new Item(100,  10, s2, "Item from another supplier");
32    }
33
34    % Helper: the existence of a Client constructor is assumed.
35    private Client newClient() {
36      return new Client(/* ... valid data ... */);
37    }
38
39    // ------------------------------------------------------------------------
40    // SUCCESS Cases
41    // ------------------------------------------------------------------------
42
43    // TC1: valid construction - all invariants satisfied.
44    @Test
45    public void tc1_validConstruction() {
46      Order o = new Order(client, "Street A", 5000);
47
48      assertEquals(o.totalQuantity(), 0);
49      assertEquals(o.cost(), 0.0, EPS);
50      assertTrue(o.itens().isEmpty());
51    }
52
53    // TC9: addItem in an empty order with a valid quantity.
54    @Test
55    public void TC9AddItemEmptyOrderValidQuantity() {
56      Order o = new Order(client, "Street A", 5000);
57
58      o.addItem(i1, 1);
59      assertTrue(o.contains(i1));
60      assertEquals(o.quantity(i1), 1);
61      assertEquals(o.cost(), 10.0, EPS);
62    }
63
64    // TC13: boundary totalQty <= 20 + floor(cost/50) (On).
65    // A unit price of 1 is used to isolate the totalQty boundary without
66    // violating the cost limit (I3).
67    // With cost = 24 * 1 = 24, the limit is
68    // 20 + floor(24/50) = 20 + 0 = 20... therefore it is necessary to raise the
69    // cost enough so that the limit reaches 24:
70    //   limit = 24  <=>  floor(cost/50) = 4  <=>  cost in [200, 250).
71    // Cost = 240 is chosen with items of price 10 (24 units * 10 = 240),
72    // keeping cost <= 1000.
73    @Test
74    public void TC13TotalQuantityAtDynamicLimit() {
75      Order o = new Order(client, "Street A", 5000);
76      // 3 items of price 10, quantities 10 + 10 + 4 = 24 units
77      Item a = new Item(150, 10, s1, "A");
78      Item b = new Item(150, 10, s1, "B");
79      Item c = new Item(150, 10, s1, "C");
80
```

```
81      o.addItem(a, 10);
82      o.addItem(b, 10);
83      o.addItem(c,  4);
84
85      assertEquals(o.totalQuantity(), 24); // = 20 + floor (240/50) = 24
86      assertEquals(o.cost(), 240.0, EPS);
87    }
88
89    // TC19: adding an item from the SAME supplier is accepted.
90    @Test
91    public void TC19SameSupplierIsAccepted() {
92      Order o = new Order(client, "Street A", 5000);
93
94      o.addItem(i1, 2);
95      o.addItem(i2, 3); // i2 also from supplier s1
96
97      assertTrue(o.contains(i1));
98      assertTrue(o.contains(i2));
99      assertEquals(o.totalQuantity(), 5);
100     assertEquals(o.cost(), 2 * 10 + 3 * 5, EPS);
101   }
102
103   // ------------------------------------------------------------------------
104   // FAILURE Cases
105   // ------------------------------------------------------------------------
106
107   // TC2: boundary addrLength >= 1 (Off) - empty address.
108   @Test(expectedExceptions = InvalidOperationException.class)
109   public void TC2EmptyAddressThrows() {
110     new Order(client, "", 5000);
111   }
112
113   // TC6: null client.
114   @Test(expectedExceptions = InvalidOperationException.class)
115   public void TC6NullClientThrows() {
116     new Order(null, "Street A", 5000);
117   }
118
119   // TC10: boundary quantity <= 10 (Off) - quantity 11.
120   @Test
121   public void TC10QuantityAboveMaxThrowsAndKeepsState() {
122     Order o = new Order(client, "Street A", 5000);
123     try {
124       o.addItem(i1, 11);
125       fail("Expected InvalidOperationException");
126     } catch (InvalidOperationException e) {
127       assertFalse(o.contains(i1)); // state unchanged
128       assertEquals(o.totalQuantity(), 0);
129     }
130   }
131
132   // TC20: adding an item from a DISTINCT supplier.
133   @Test(expectedExceptions = InvalidOperationException.class)
134   public void TC20DifferentSupplierThrows() {
135     Order o = new Order(client, "Street A", 5000);
136     o.addItem(i1,     1); // supplier s1
137     o.addItem(iOther, 1); // supplier s2 -> exception
138   }
139 }
```

Listing 1: Skeleton of the TestNG test class for Order.